

PCP — Patrones para el parcial

Armado a partir del simulacro, las guías 1–4, las prácticas 1–4, las teóricas 1–5 y los labos 1 y 3.

Cómo usar este apunte si arrancás de cero: esto es un *machete*, no una explicación. Primero leé el resumen del tema (`knowledge/resumenes/0X-*.md`) y la práctica correspondiente. Después volvé acá para ver "qué patrón aplica cuando el enunciado dice X". Lo ideal es terminar intentando el simulacro con este machete al lado.

Glosario mínimo

Término	Qué es, en una línea
Thread	un hilo de ejecución; varios comparten la memoria
Interleaving	el orden en que se mezclan las instrucciones de los threads (cualquiera es posible)
Race condition	el resultado depende de ese orden (p. ej. se pierde un <code>x++</code>)
Sección crítica (SC)	el pedazo de código donde tiene que haber a lo sumo un thread a la vez
Deadlock	todos esperan algo que nunca va a pasar
Inanición (starvation)	un thread en particular espera para siempre mientras los demás avanzan
Busy-waiting	esperar girando en un <code>while</code> y gastando CPU
Semáforo	contador de permisos: <code>acquire</code> toma uno (o te duerme), <code>release</code> devuelve uno (o despierta a alguien)
Débil / fuerte	despierta a cualquiera / despierta en orden FIFO
Monitor	objeto cuyos métodos corren de a uno; permite <code>wait</code> / <code>signal</code> sobre condiciones
Mesa / Hoare	al hacer <code>signal</code> : sigue el que avisó / pasa ya mismo el despertado
Lock	candado: <code>lock</code> / <code>unlock</code>
CAS	<code>compareAndSet</code> (esperado, nuevo) atómico, la base de lo lock-free
Linealizable	cada operación "pasa de golpe" en un instante dentro de su intervalo
Wait-free / lock-free	todos terminan / siempre termina alguno
SC / TSO	consistencia secuencial (el modelo ideal) / el modelo real de x86, que reordena Store→Load

0. Formato del parcial (según el simulacro)

Ej	Tema	Qué te piden siempre
1	Exclusión mutua con busy-waiting + modelo de memoria	(a) romperlo sin atomicidad, (b) analizarlo con atomicidad, (c) ¿vale en Java/hardware real?
2	Semáforos	(a) modelar y resolver, (b) inanición / sobrepasos con semáforos débiles, (c) una variante ("¿qué cambia si...?")

Ej	Tema	Qué te piden siempre
3	Monitores (Mesa, colas no fair, sin spurious wakeups)	(a) resolver, (b) "¿qué cambia con Hoare?"
4	Estructuras concurrentes	(a) granularidad fina + linealizabilidad, (b) progreso, (c) versión CAS lock-free, (d) analizar un código dado, (e) ¿wait-free?

Para aprobar: 2 ejercicios B + 1 R, y **uno de los B tiene que ser el de semáforos o el de monitores**. → Semáforos y monitores son la prioridad nº 1.

Supuestos por defecto (usalos y escribilos): semáforos **débiles**; monitores **Mesa** (signal-and-continue); colas de condición **no FIFO**; sin spurious wakeups. Todo código tiene que estar **justificado** (es obligatorio).

1. Ejercicio 1 — Algoritmos de exclusión mutua

1.1 Las 3 propiedades (revisalas siempre, en este orden)

1. **Exclusión mutua** (safety): nunca hay 2 en la SC. Contraejemplo = traza que llega a (p en SC, q en SC).
2. **Sin deadlock/livelock** (liveness): si varios quieren entrar, alguno entra. Contraejemplo = traza donde todos esperan para siempre (livelock si están haciendo busy-waiting).
3. **Sin inanición** (liveness): todo el que quiere entrar, entra. Contraejemplo = **traza infinita y fair** (toda instrucción continuamente habilitada se termina ejecutando) donde uno nunca entra. Suele ser un ciclo en el grafo de estados.

Supuestos del modelo: la SC siempre termina; **la SNC puede no terminar** (así cae la alternancia estricta).

1.2 Catálogo de fallas típicas (qué buscar en la propuesta)

Qué tiene la propuesta	Qué suele romper	Traza tipo
<code>while(flag); flag = true</code> (primero chequeo, después marco)	Mutex	los dos pasan el while antes de que alguno marque
<code>flag[id] = true; while(flag[otro])</code> (primero marco, después chequeo)	Deadlock/livelock	los dos marcan y se quedan esperando
Turno estricto <code>while(turno != id)</code>	Inanición	el otro se queda en la SNC y nunca pasa el turno
<code>x++ / cant++</code> sin atomicidad	Update perdido → mutex o deadlock	desagregar en <code>tmp = x; x = tmp + 1</code> y entrelazar
Ticket que al salir decrementa el ticket en vez de avanzar el turno (G1 ej10, ej14)	Mutex (dos threads con el mismo ticket)	A saca 0, B saca 1, A sale (ticket vuelve a 1), C saca 1 → B y C con 1
Variable de turno (<code>proximo</code> , <code>actual</code>) que no se resetea al salir el último	Mutex (queda un valor viejo que deja pasar a alguien)	ver simulacro abajo
Se elige "el de menor id"	Inanición de los ids altos	0 y 1 alternan para siempre y 2 nunca entra
Peterson generalizado con <code>otro = (id+1)%n</code> (G1 ej11)	Mutex con $n > 2$	cada uno mira a un solo vecino

Qué tiene la propuesta	Qué suele romper	Traza tipo
Bakery sin desempate <code>j < id</code> (G1 ej13)	Deadlock con números iguales (el mutex vale)	los dos sacan el mismo número y se esperan
Función no atómica que recorre un array (<code>algunVerdadero</code> , <code>llamarProximo</code>)	Entrelazado dentro del for	uno lee <code>flag[i]</code> antes de que el otro lo escriba
Dos operaciones atómicas separadas (<code>soyPrimero = anotarse()</code> y después <code>llamarProximo()</code>)	Check-then-act entre las dos	otro se mete entre ambas

Simulacro, inciso (b): incluso con las funciones atómicas, `proximo` no se resetea. Traza: T1 sale último (`proximo = 1` queda viejo) → T0 hace `anotarse` (es primero) → T1 hace `anotarse` (no es primero), ve `proximo == 1` y **entra** → T0 hace `llamarProximo` → `proximo = 0` → T0 **entra**. Se rompe el mutex. Además hay inanición porque siempre se elige el menor id.

1.3 Cómo escribirlo

- **Traza como tabla:** columnas T1 | T2 | Estado (con las variables locales, p. ej. `p.tmp = 0`). Marcá dónde está cada PC.
- **"Se cumple X":** no alcanza con una traza. Tenés que dar un **invariante** o argumentar por el absurdo ("supongamos que los dos están en la SC; entonces el último en entrar vio...").
- **"Si fuera atómica":** reagrupá cada llamada como un solo paso y rehacé el análisis. Lo típico es que el mutex pase a valer y quede la inanición, o que aparezca un deadlock.

1.4 Inciso (c): modelo de memoria (respuesta modelo)

- Todo el análisis anterior supone **Consistencia Secuencial** (un interleaving que respeta el orden de programa).
- **Hardware (x86-TSO):** el *store buffer* permite reordenar **Store→Load** entre direcciones distintas: "escribo mi flag y después leo el del otro" puede leer un valor viejo (Dekker litmus: `r1 == r2 == 0`). ARM/POWER relajan todavía más (LL, LS, SS, SL).
 - Restricción mínima: un **FENCE StoreLoad** (`mfence`) entre la escritura y la lectura posterior, o usar un RMW atómico (`xchg`, `lock cmpxchg`, `lock xadd`, que en x86 funcionan como barrera). En ARM: `dmb` o `acquire/release`.
- **Lenguaje (Java/JMM):** si hay variables compartidas sin sincronizar hay **data races**, y la JMM solo garantiza SC para programas **DRF** (DRF-SC). Además el JIT puede cachear `proximo` en un registro, y entonces `while(proximo != id);` **no termina nunca** (visibilidad). También puede reordenar.
 - Restricción mínima: marcar las compartidas **volatile** (escritura volatile *happens-before* toda lectura posterior de esa variable: da visibilidad y prohíbe reordenar). `volatile boolean[]` solo hace volátil la **referencia**, no los elementos → hay que usar `AtomicIntegerArray` (o `AtomicBoolean[]`). Para que las funciones sean atómicas en Java hace falta `synchronized / Lock`, que ya dan *happens-before*.
- **Respuesta por propiedad:**
 - Las que **valían** en (b) se mantienen solo si agregás esas restricciones (volatile/fence).
 - Las que **fallaban** en (b) **siguen fallando por el mismo motivo**: toda ejecución SC también es una ejecución TSO/JMM válida ($SC \subseteq TSO$), así que el contraejemplo sigue siendo posible. Encima aparecen **motivos adicionales** (reordenamientos, falta de visibilidad, spinning infinito). Las garantías de la arquitectura **nunca arreglan** algo que ya falla bajo SC.

Baja prioridad (no apareció en el simulacro): semántica denotacional, diagramas de estados completos, contar trazas (G1 ej1–4).

2. Ejercicio 2 — Semáforos

2.1 Del enunciado al patrón (lo más importante del apunte)

Si el enunciado dice...	Patrón
"A tiene que pasar antes que B"	Señal: <code>sem(0)</code> , release después de A, acquire antes de B
"los dos se esperan" / "se encuentran"	Rendezvous: cada uno libera el suyo y toma el del otro
"a lo sumo K a la vez"	Multiplex <code>sem(K)</code>
"que alternen" / "que difieran en a lo sumo 1"	Split semaphores (<code>sem(1)</code> / <code>sem(0)</code> , o <code>1 / 1</code>)
"#F ≤ #A"	Semáforo contador en 0: A hace release, F hace acquire
buffer / tareas / cola	Productor-consumidor: <code>notEmpty(0)</code> , <code>notFull(N)</code> , <code>mutexP</code> , <code>mutexC</code>
"varios del mismo tipo juntos, excluyendo a otro tipo"	Lightswitch (uno por tipo) sobre un <code>room</code> compartido
"a X no le molesta esperar" / "Y no puede esperar por un X que todavía no empezó"	Prioridad para Y (lectores): lightswitch de Y y sin turnstile. X puede sufrir inanición, y lo aceptás (justificalo con el enunciado)
"nadie espera para siempre" / "sin inanición para ambos"	Lightswitch + turnstile FUERTE compartido
"si hay un X esperando, los nuevos Y esperan"	Prioridad X: X retiene el turnstile, o lightswitch de X sobre <code>leer</code> (escritores)
"esperar a que lleguen N" / rondas / tramos	Barrera reutilizable (2 turnstiles)
"necesita 2+ recursos distintos a la vez"	Orden global de adquisición (menor id primero), o limitar a $N-1$, o romper la simetría
"necesita k unidades del mismo pool"	Tomarlas bajo un mutex (o <code>acquire(k)</code>). Si las tomás de a una → deadlock
"en orden de llegada" / "no se pasan" / FIFO sin semáforos fuertes	Paso de testigo: cola de semáforos privados (<code>FilaDeSalida</code>)
"si está lleno, se va"	Contador bajo mutex. Nunca <code>availablePermits()</code> (check-then-act)
"necesita la combinación exacta de 2 cosas" (fumadores)	Threads gestores + booleanos bajo mutex (los acquire sueltos hacen deadlock)
"compra solo si lo que leyó sigue vigente"	Lectores-escriitores + número de versión (<code>VentaTickets</code>)

El caso que decías: el simulacro (banco) es *lightswitch de transferencias + room*, y el informe hace de "escritor". La frase "no tolera que un informe que todavía no empezó haga esperar a una transferencia" significa **prioridad lectores, sin turnstile**. La variante esperable es "el informe no puede esperar para siempre" (→ agregás un **turnstile fuerte**) o "si hay un informe esperando, las transferencias nuevas esperan" (→ **prioridad escritores**). Para las cuentas: `cuenta[min]` y después `cuenta[max]` .

2.2 Plantillas

```
// LIGHTSWITCH (uno por "tipo"; room compartido = Semaphore(1))
encender(room): m.acquire(); c++; if (c == 1) room.acquire(); m.release()
apagar(room): m.acquire(); c--; if (c == 0) room.release(); m.release()

// PRIORIDAD LECTORES (el escritor puede sufrir inanición)
lector: ls.encender(room); leer; ls.apagar(room)
escritor: room.acquire(); escribir; room.release()

// SIN INANICIÓN (turnstile FUERTE: Semaphore(1, true))
lector: turnstile.acquire(); turnstile.release(); ls.encender(room); leer; ls.apagar(room)
escritor: turnstile.acquire(); room.acquire(); turnstile.release(); escribir; room.release()

// PRIORIDAD ESCRITORES (noReaders = leer, noWriters = escribir)
lector: mutexP.acquire(); noReaders.acquire(); lsL.encender(noWriters);
        noReaders.release(); mutexP.release(); leer; lsL.apagar(noWriters)
escritor: lsE.encender(noReaders); noWriters.acquire(); escribir;
        noWriters.release(); lsE.apagar(noReaders)

// BARRERA REUTILIZABLE
mutex.acquire(); count++; if (count == n) t1.release(n); mutex.release(); t1.acquire()
mutex.acquire(); count--; if (count == 0) t2.release(n); mutex.release(); t2.acquire()

// VARIOS RECURSOS: orden total (sin espera circular)
a = min(i, j); b = max(i, j); r[a].acquire(); r[b].acquire(); ...; r[b].release(); r[a].release()

// PASO DE TESTIGO (orden FIFO con semáforos débiles)
anotarse(): mutex.acquire(); s = Semaphore(0); primero = fila.vacia(); fila.addLast(s);
        mutex.release(); if (primero) s.release(); return s
// el thread hace s.acquire() cuando le toca actuar; al terminar:
avisar(): mutex.acquire(); fila.removeFirst(); sig = fila.peekFirst(); mutex.release();
        if (sig != null) sig.release()
```

Variantes de la guía que conviene tener vistas:

- **Baño (G2 ej11):**
 - (a) lightswitch de personas sobre vacío + multiplex(8).
 - (b) la limpieza retiene un turnstile.
 - (c) **mover el encender después del multiplex** hace que los que esperan toilette no cuenten para la limpieza.
 - Lección: **dónde ponés el lightswitch define quién cuenta.**
- **Puente (G2 ej12 / Saldungaray):** (a) un lightswitch por sentido; (b) + multiplex(3) adentro; (c) inanición → turnstile fuerte.
- **Gimnasio (G2 ej7):** aparato = sem(1), discos = sem(D). Tomar discos de a uno reteniendo el aparato → deadlock; se arregla con un mutex para tomar los k discos.
- **Filósofos:** sillas(N-1), o que un filósofo tome los tenedores al revés.

2.3 Plantilla de justificación (escribila siempre)

1. **Recursos y agentes:** qué es cada semáforo y **qué invariante cumple** ($\#enSC + S.V = 1$, $notEmpty.V \leq \#buffer$, ...).
2. **Sin race conditions:** toda variable compartida se lee y escribe con su mutex tomado.

3. **Sin deadlock:** no hay espera circular (orden total de adquisición); nadie espera algo que solo libera quien lo está esperando; todo adquiere tiene su release en todos los caminos.
4. **Inanición:** con semáforos débiles **cualquiera puede ser sobrepasado sin cota** (barging). Mostralo con una traza: T espera en `s.acquire()`; se libera `s`; llega T' y se lo gana; se repite para siempre.
 - Con un semáforo **fuerte**, la cota es la cantidad de threads que ya estaban delante en la cola ($\leq N-1$).
 - El apunte de la práctica 3 lo avisa explícitamente: *"formalmente el turnstile tiene que ser `new Semaphore(1, true)` si piden justificar no-starvation con precisión (Ejem, un parcial, ejem)"*.
5. **Variante (inciso c):** decí qué cambia y **si la prueba de no-deadlock sigue valiendo**. Ejemplo: con destinos múltiples, ordenás **todas** las cuentas y las tomás de menor a mayor, y el argumento de orden total sigue valiendo.

3. Ejercicio 3 — Monitores

3.1 Reglas de Mesa (las que más se evalúan)

- `while (!cond) wait(c);` **siempre while:** el despertado compite con los que recién entran ($E = W < S$) y la condición puede haber cambiado.
- **Señalizá siempre que cambia el estado.** Condicionar el signal a una transición puntual (`if (cant == 1) signal`) provoca un **lost wake-up** (teórica 3).
- `signal` alcanza solo si los que esperan son **intercambiables** (pool de tareas). Si no lo son (distintos motivos o tickets distintos en la misma cola), o si se liberan varios $\rightarrow$ `signalAll`.
- En Java con `synchronized` hay **una sola cola** $\rightarrow$ con varias condiciones lógicas usá `notifyAll` (o `ReentrantLock + varias Condition`).
- **Mesa no es FIFO** y las colas no son fair: si piden "tarde o temprano todos...", **necesitás imponer el orden vos** (ticket).

3.2 Del enunciado al patrón

Enunciado	Patrón
"tarde o temprano todos consiguen..." + pedidos heterogéneos (mesas de k lugares, tomar k recursos: G3 ej10c $\approx$ simulacro ej3)	Ticket FIFO: solo avanza el primero de la fila
barrera / "hasta que lleguen N"	Contador + fase (generación)
turnos cíclicos (secuenciador ternario)	<code>turno = (turno + 1) % 3 + signalAll</code>
"liberar N a la vez" (atrapador)	contador de "pases" + generación (para que no se cuele uno que no esperó)
cliente/servidor (peluquería, secretaria)	varias condiciones (<code>hayCliente</code> , <code>listo</code> , <code>terminó</code>) + ticket si importa el orden
condición compuesta (pizza grande o 2 chicas; lapicera y cuaderno; filósofos)	un booleano/contador por recurso y un <code>while</code> con la condición completa
fases de un recurso (charla, bote)	variable de estado (<code>ESPERANDO</code> / <code>EN_CURSO</code> / <code>BAJANDO</code> , <code>costa actual</code> , ...)
semáforo fair con monitor	pasar el testigo: si hay alguien en la cola, en vez de <code>sv++</code> hacés <code>signal</code> (con <code>empty(c)</code>)

3.3 Plantilla: ticket (resuelve el simulacro)

```
monitor Juegos {
    bool libre[M] = {true, ...}; int proximoTicket = 0, turno = 0; condition cambio
    int sentarse(k) {
        int t = proximoTicket++
        while (t != turno || mesaLibreQueEntra(k) == -1) wait(cambio)
        int m = mesaLibreQueEntra(k) // la más chica donde entra
        libre[m] = false; turno++
        signalAll(cambio) // ahora el siguiente ticket puede intentar
        return m
    }
    void levantarse(m) { libre[m] = true; signalAll(cambio) }
}
```

- **Sin inanición:** solo se sienta el ticket `turno`. Su mesa se libera tarde o temprano (todo grupo sentado se va) y nadie se la puede sacar. Cada ticket llega a ser `turno` después de una cantidad finita de grupos.
- **Sin deadlock:** el primero de la fila solo espera mesas ocupadas por grupos que se van a ir.
- **Costo** (conviene decirlo): *head-of-line blocking*. Si querés más paralelismo, podés dejar pasar a otros mientras no le tomen la mesa al primero o con una cota de sobrepasos, pero tenés que justificarlo.

3.4 Inciso "¿y con Hoare?" (respuesta modelo)

- Hoare/IRR: $E < S < W$. El despertado corre **inmediatamente** con el lock, así que la condición que garantizó el que señalizó **sigue siendo verdadera**. → Los `while` pasan a ser `if`.
- Se puede **pasar el recurso directamente**: el que señala deja asignado el estado para el despertado (p. ej. marca la mesa como tomada para él antes del signal). Nadie se lo puede robar, porque los nuevos (E) tienen menor prioridad.
- Con Hoare, `signal` suspende al que señala (va a la cola urgente) → conviene que `signal` sea lo último. `signalAll` se reemplaza por **signal en cascada** (cada despertado despierta al siguiente).
- Si además las colas de condición son FIFO (Hoare clásico), el orden de la cola puede reemplazar al ticket.
- Guía 3 ej1 es este mismo tipo de pregunta (signal y espera urgente vs signal y continúa, con una traza).

4. Ejercicio 4 — Estructuras concurrentes / lock-free

4.1 Definiciones que tenés que poder escribir

- **Linealizable:** cada operación parece ocurrir en un instante (su **punto de linealización**, PL) entre la invocación y la respuesta, y ordenar por PL da una historia secuencial válida. Respetar el orden de tiempo real y es **composicional**. La consistencia secuencial no respeta el tiempo real y **no** es composicional.
- **Progreso:**
 - **wait-free** $\subset$ **lock-free** $\subset$...
 - Wait-free: todos terminan en finitos pasos, sin importar los demás.
 - Lock-free: siempre termina **alguno**.

- Bloqueantes (asumen scheduler fair): **starvation-free** (todos terminan) $\subset$ **deadlock-free** (alguno termina).

4.2 Recetas

- **Granularidad fina sobre un arreglo:** un lock por celda.
 - Operación sobre 1 celda: toma solo ese lock (PL = la escritura).
 - Operación sobre todas (get/suma): toma **todos los locks en orden creciente**, lee y libera. El PL es el instante en que tiene todos tomados, que es un snapshot real.
 - Sin deadlock: todos toman en el mismo orden total.
 - Sin inanición: **solo si los locks son fair** (`ReentrantLock(true)`); `synchronized` no es fair.
- **Lista:** hand-over-hand desde `head`. El orden de los nodos es el orden total que evita el deadlock.
- **Varios campos independientes (Figura):** un lock por grupo (`lockPos`, `lockTam`). Una operación que toca los dos $\rightarrow$ siempre `lockPos` antes que `lockTam`.
 - Versión lock-free: objeto **inmutable** (`x`, `y`) en un `AtomicReference`, y CAS del objeto entero.
- **Swap(i, j):** lock `min(i, j)` y después `max(i, j)`. Ojo con `i == j`.
- **Optimista (G4 ej5):** lees, computas lo costoso **sin lock**, tomás el lock y **validás** (valor o versión sin cambiar). Si cambió, reintentás. Es deadlock-free pero **no** starvation-free.
- **CAS loop (inc lock-free):**

```
do { v = a.get(i); } while (!a.compareAndSet(i, v, v + 1)); // AtomicIntegerArray
```

- PL: el CAS exitoso.
- **Lock-free** (un CAS solo falla si otro tuvo éxito); **no wait-free** (puede fallar infinitas veces).
- **Double collect** (el `get` del simulacro):
 - Si los valores solo **crecen**, `total1 == total2` implica que ninguna celda cambió entre las dos lecturas, así que es un snapshot real y es **linealizable**.
 - **No es wait-free:** con incrementos continuos puede no terminar nunca.
 - Si hay decrementos o `reset`, sumas iguales **no** garantizan nada (+1 en una celda y -1 en otra). Hay que comparar **arreglo contra arreglo**, y con ABA hacen falta **versiones/sellos** por celda.
 - Dato fino: para un contador **monótono**, hasta el collect simple (sin locks ni reintento) es linealizable, porque el total pasa por todos los enteros entre el inicio y el fin. Además es wait-free.
- **Híbrido (get con locks, inc con CAS):** los locks no frenan a los CAS, así que el "snapshot con locks" no protege nada y queda igual que un collect simple.
- **ABA sin GC** (pila o cola lock-free): un nodo reciclado vuelve a la misma dirección y el CAS tiene éxito cuando no debería. Se arregla con `AtomicStampedReference` (sello o versión que se incrementa en cada CAS).

4.3 Tabla de listas (teórica 4 + labo 3)

Impl.	contains	PL de add/remove exitosos	Progreso
Gruesa	con lock	escritura de <code>pred.next</code> (bajo lock)	deadlock-free; sin inanición si el lock es fair
Fina (hand-over-hand)	con locks	escritura de <code>pred.next</code>	deadlock-free (orden head $\rightarrow$ tail); sin inanición si los locks son fair

Impl.	contains	PL de add/remove exitosos	Progreso
Optimista	con locks + validate (recorre dos veces)	escritura, después de una validación exitosa	deadlock-free; con inanición aunque los locks sean fair
Lazy (marked)	wait-free , sin locks	remove: al setear <code>marked</code> ; add: <code>pred.next</code>	add/remove con inanición
Lock-free (AtomicMarkableReference)	wait-free	add: CAS de <code>pred.next</code> ; remove: CAS que marca	add/remove lock-free

Colas:

- Dos locks: deadlock-free, sin inanición si los locks son fair. PL del `enq` en `tail.next = e`.
- Michael-Scott lock-free: con *helping*. PL del `enq` en el CAS de `last.next` ; PL del `deq` en el CAS de `head`.
- Pila lock-free: PL en el CAS de `top`.

5. Qué priorizar (en orden)

1. **Semáforos:** que salgan de memoria el lightswitch, el turnstile, las dos prioridades de lectores/escritores, la barrera reutilizable, el orden de adquisición y el paso de testigo. Y la **justificación de inanición con semáforos débiles**.
2. **Monitores Mesa:** `while`, lost wake-up, cuándo `signal` y cuándo `signalAll`, **ticket FIFO** y la pregunta de **Hoare**.
3. **Ej 1:** catálogo de fallas + trazas en tabla + la respuesta modelo del inciso (c).
4. **Ej 4:** orden de locks, PL, CAS loop, double collect, wait-free vs lock-free.
5. **Casi no rinde:** semántica denotacional, contar trazas, detalles de skip lists, elimination stack/exchanger, colas duales, AQS/futex, thread pool/promesas.